

Class 7: Machine Learning 1

Mayla (PID: A19136116)

Table of contents

Background	1
K-means clustering	1
Hierarchical Clustering	8
Principal Component Analysis (PCA)	10
Analysis of UK food data	10
Data Import	10
Tidy the data	16
Exploratory analysis	16
PCA to the rescue	16

Background

Today we will explore some core machine learning methods that are very popular in bioinformatics. These include: **clustering** and **dimensionality reduction**.

K-means clustering

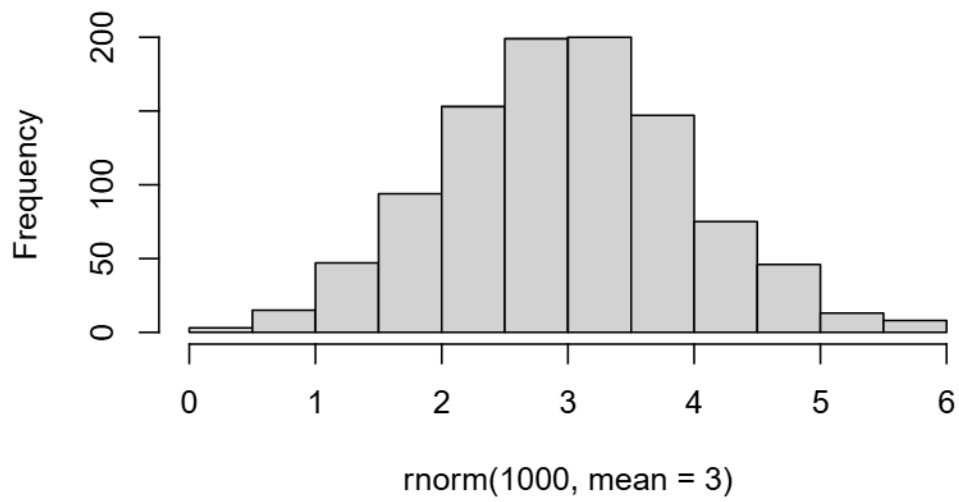
The main function in “base” R for K-means clustering, is called `kmeans()`

Before we go too deep let’s make up some “simple” data that we can cluster and know if we are getting a good answer or not. To help us do this, we can use `rnorm()`‘ function:

```
hist( rnorm(1000, mean = 3) )
```

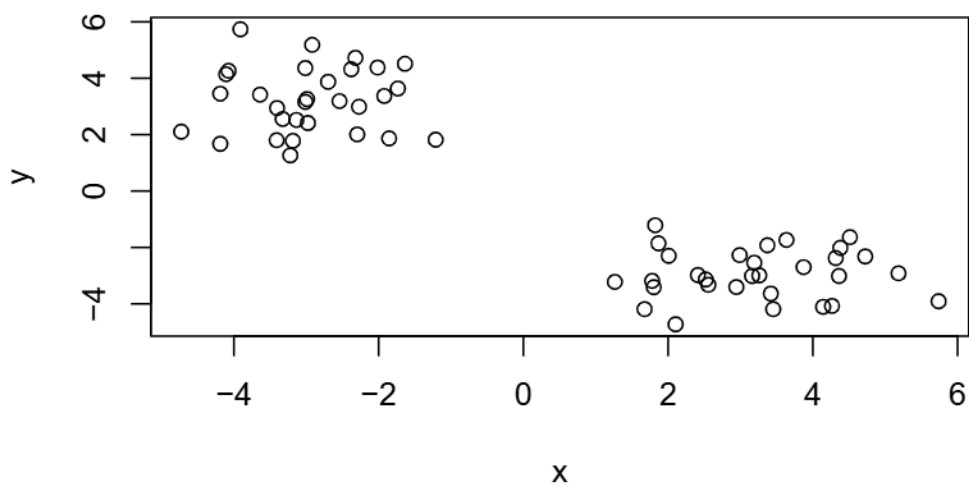


Histogram of rnorm(1000, mean = 3)



```
x <- c( rnorm(30, mean = -3), rnorm(30, mean = +3) )
```

```
z <- cbind( x = x, y = rev(x) )  
plot(z)
```



```
#cbind() / rbind() combines by column/rows
```

```
p <- 1:5
```

```
cbind(p, rev(p))
```

```
      p
[1,] 1 5
[2,] 2 4
[3,] 3 3
[4,] 4 2
[5,] 5 1
```

```
rbind(p, rev(p))
```

```
      [,1] [,2] [,3] [,4] [,5]
p      1    2    3    4    5
      5    4    3    2    1
```

Now we can run `kmeans(0)` on this input `z` and see what the results look like.

km\$cluster

[illegible]

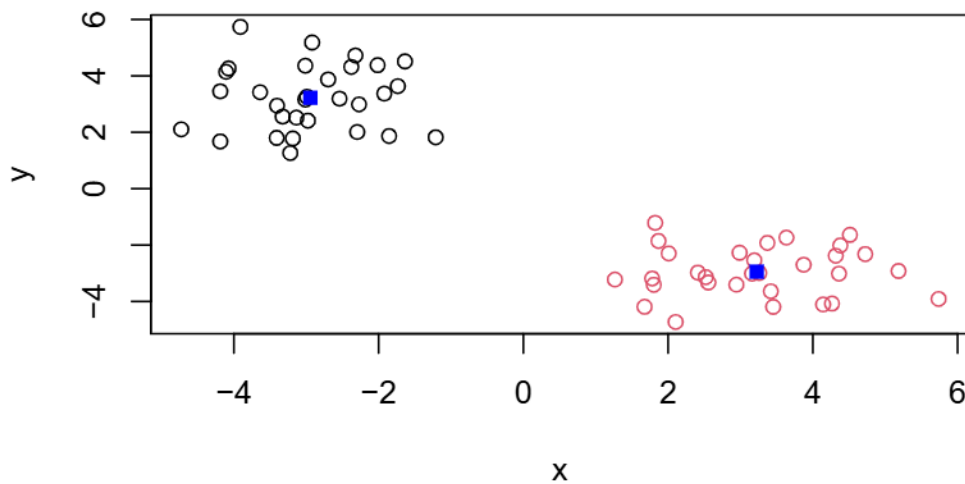
Q. What “component of your result object details cluster center?”

km\$centers

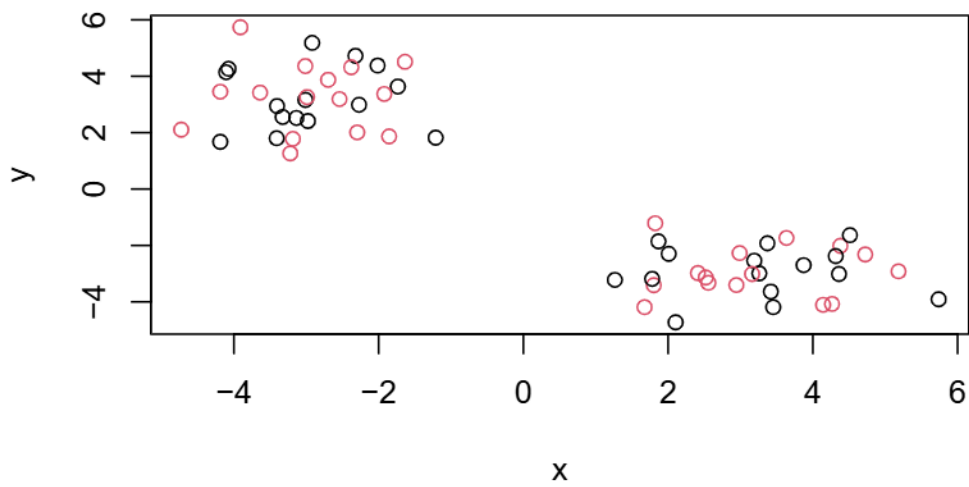
	x	y
1	-2.943496	3.224196
2	3.224196	-2.943496

Q. Plot $\mathbf{z}$ colored by the kmeans cluster assignment and add cluster centers as blue points.

```
plot(z, col=km$cluster)
points(km$centers, col= "blue", pch = 15)
```



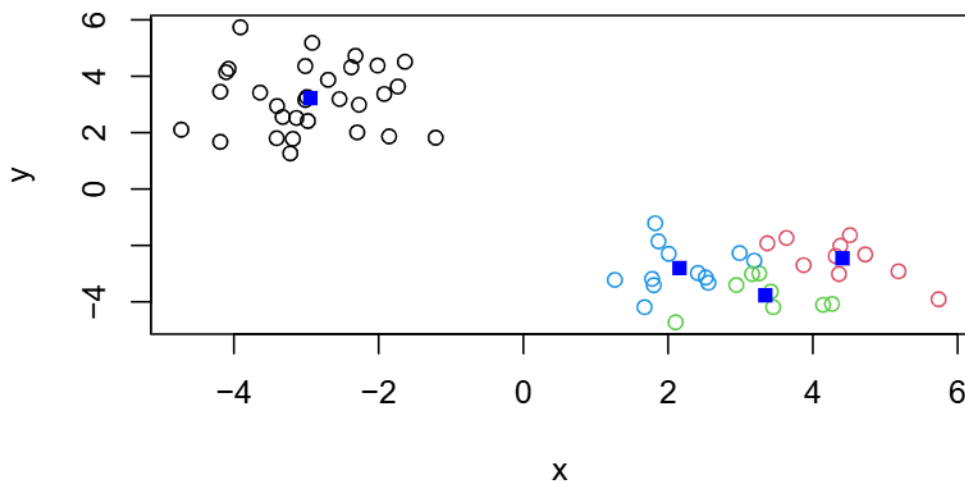
```
plot(z, col= c(1,2))
```



Q. Run a K-means clustering and plot the results asking for 4 clusters?

```
km4 <- kmeans(z, centers = 4)
plot(z, col=km4$cluster)
points(km4$centers, col= "blue", pch = 15)
```

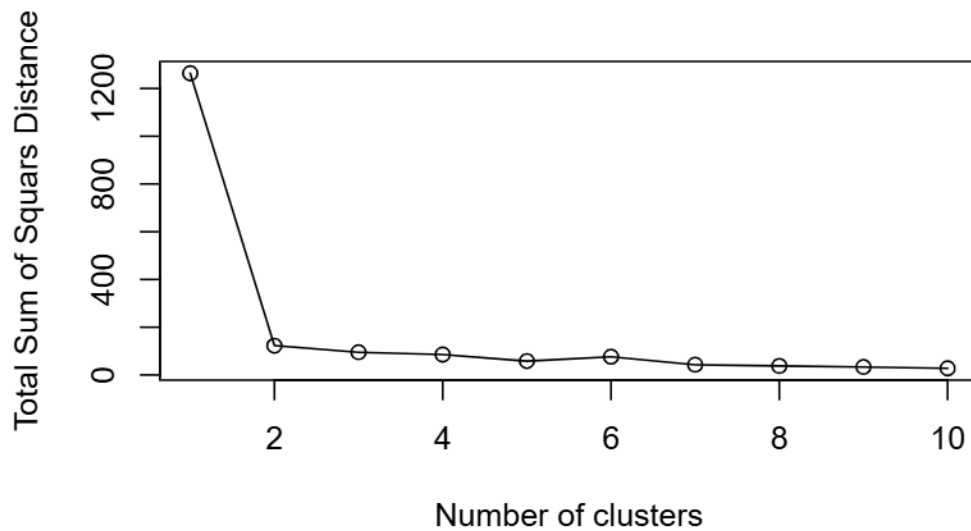




N.B. You need to tell K-means the number of clusters (i.e. set `centers=2`)!!

One approach is to try different values for `centers` and then pick the best...

```
ans <- NULL
for(i in 1:10) {
  km <- kmeans(z, centers = i)
  ans <- c(ans, km$tot.withinss)
}
plot(ans, type = "o",
      xlab = "Number of clusters",
      ylab = "Total Sum of Squares Distance" )
```



Hierarchical Clustering

The main function in “base” R for Hierarchical Clustering is called `hclust()`

This function does not take your “raw” data for clustering. You must first build a “distance matrix” from your data and pass this as input to `hclust()`

```
d <- dist(z)
hc <- hclust(d)
hc
```

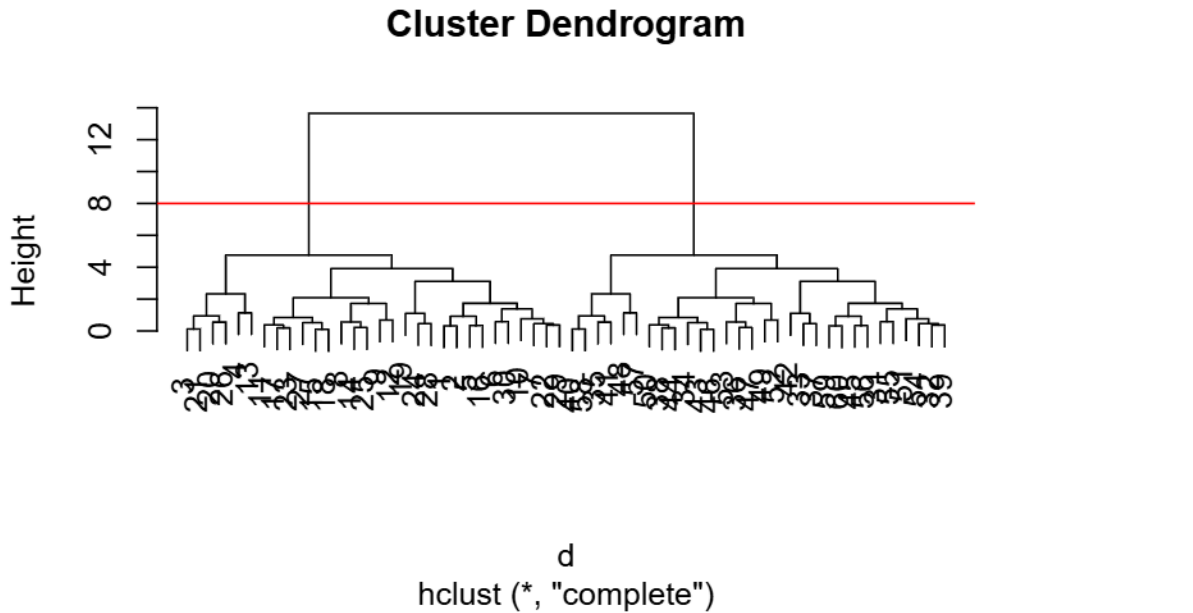
Call:

```
hclust(d = d)
```

```
Cluster method : complete
Distance       : euclidean
Number of objects: 60
```

There is a bespoke `plot()` method for `hclust()` result objects.


```
plot(hc)
abline(h=8, col="red")
```



Once we have our `hclust()` object (our “tree” of cluster dendrogram) we can “*cut*” the tree to reveal the clustering pattern.

```
cutree(hc, h=8)
```

[illegible]

```
cutree(hc, k=4)
```

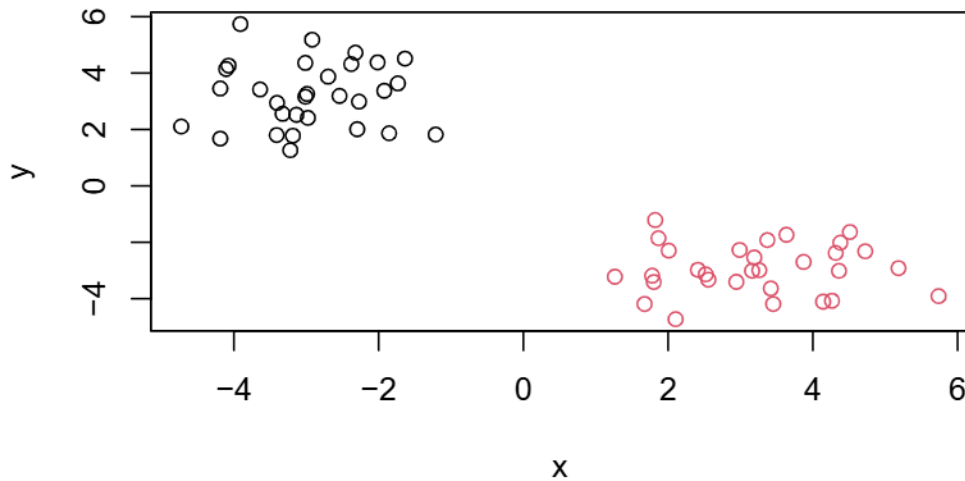
[1] 1 1 2 2 1 1 1 1 1 1 1 2 1 1 1 1 1 2 2 1 1 1 1 2 1 1 1 1 3 3 3 3 4 3 3
[39] 3 4 4 3 3 3 3 3 4 3 3 3 3 3 3 3 3 4 4 3 3

```
#cuts in a way to get 4 groups
```

Q. Make a plot of z with your hclust results (i.e. colored by cluster membership)



```
grps <- cutree(hc, k=2)
plot(z, col = grps)
```



Principal Component Analysis (PCA)

PCA is a dimensionality reduction method that is popular for revealing patterns in complex datasets.

Analysis of UK food data

Let's look at some data on the eating habits of folks from the UK to see if there are patterns and trends that have the same regions being distinct from others.

Data Import

The data is made available in CSV format so we can use the `read.csv()` function for the import to R:

```
r1 <- "https://tinyurl.com/UK-foods"
x <- read.csv(r1)
```

```
row.names(x) <- x[, 1]
x <- x[, -1]
x
```

	England	Wales	Scotland	N.Ireland
Cheese	105	103	103	66
Carcass_meat	245	227	242	267
Other_meat	685	803	750	586
Fish	147	160	122	93
Fats_and_oils	193	235	184	209
Sugars	156	175	147	139
Fresh_potatoes	720	874	566	1033
Fresh_Veg	253	265	171	143
Other_Veg	488	570	418	355
Processed_potatoes	198	203	220	187
Processed_Veg	360	365	337	334
Fresh_fruit	1102	1137	957	674
Cereals	1472	1582	1462	1494
Beverages	57	73	53	47
Soft_drinks	1374	1256	1572	1506
Alcoholic_drinks	375	475	458	135
Confectionery	54	64	62	41

Q1. How many rows and columns are in your new data frame named x?
What R functions could you use to answer this questions?

```
nrow(x)
```

```
[1] 17
```

```
ncol(x)
```

```
[1] 4
```

```
# Preview the first 6 rows
```

```
x <- read.csv(rl, row.names=1)
head(x)
```

	England	Wales	Scotland	N.Ireland
Cheese	105	103	103	66
Carcass_meat	245	227	242	267
Other_meat	685	803	750	586
Fish	147	160	122	93
Fats_and_oils	193	235	184	209
Sugars	156	175	147	139

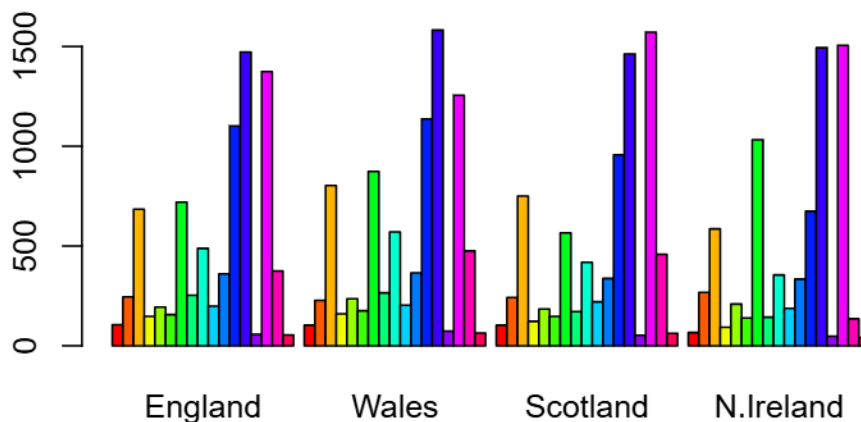
Q2. Which approach to solving the ‘row-names problem’ mentioned above do you prefer and why? Is one approach more robust than another under certain circumstances?

I prefer `read.csv(url, row.names = 1)` because it is cleaner and fixes the row names during import. It is better when you already know the first column should be row names. The manual method is useful if the file was already read incorrectly and needs to be fixed afterward.

Q3. Changing what optional argument in the above `barplot()` function results in the following plot?

The optional argument changed is `beside`, from `TRUE` to `FALSE` (or removed). Setting `beside = FALSE` produces a stacked barplot instead of side-by-side bars.

```
# Using base R
barplot(as.matrix(x), beside = T, col = rainbow(nrow(x)))
```



Q4. Changing what optional argument in the above `ggplot()` code results in a stacked barplot figure?

```
library(tidyr)

# Convert data to long format for ggplot with `pivot_longer()`
x_long <- x |>
  tibble::rownames_to_column("Food") |>
  pivot_longer(cols = -Food,
               names_to = "Country",
               values_to = "Consumption")

dim(x_long)
```

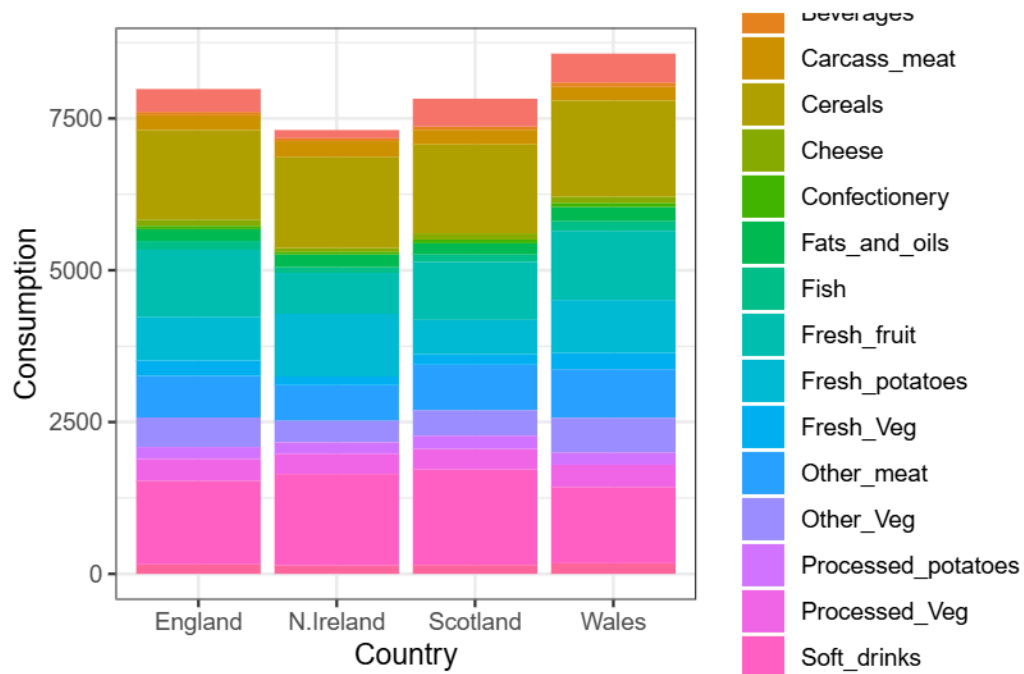
```
[1] 68  3
```

```
head(x_long)
```

```
# A tibble: 6 x 3
  Food          Country Consumption
<chr>         <chr>         <int>
1 "Cheese"      England          105
2 "Cheese"      Wales            103
3 "Cheese"      Scotland         103
4 "Cheese"      N.Ireland         66
5 "Carcass_meat " England          245
6 "Carcass_meat " Wales            227
```

```
library(ggplot2)
```

```
ggplot(x_long) +
  aes(x = Country, y = Consumption, fill = Food) +
  geom_col(position = "stack") +
  theme_bw()
```

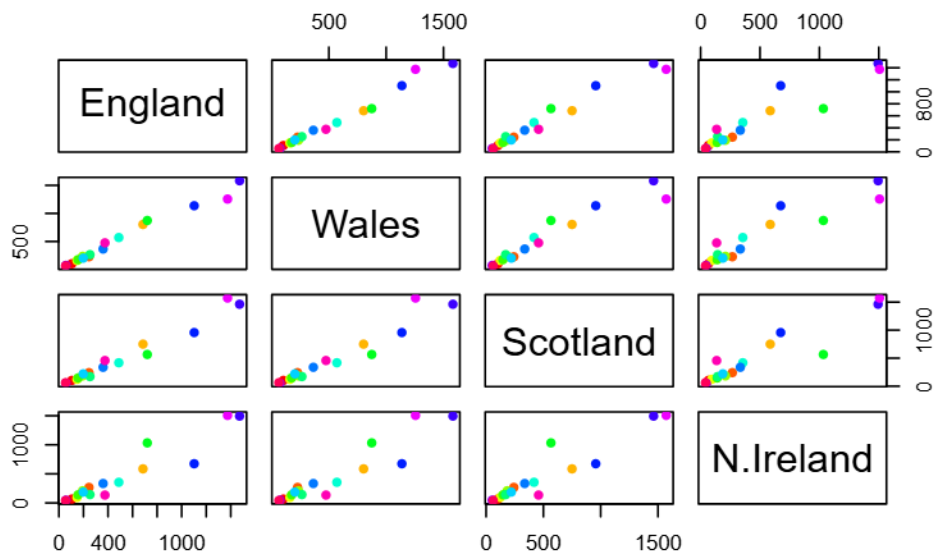


The optional argument changed is position from “dodge” to “stack”, which creates a stacked barplot instead of grouped bars.

Q5. We can use the `pairs()` function to generate all pairwise plots for our countries. Can you make sense of the following code and resulting figure? What does it mean if a given point lies on the diagonal for a given plot?

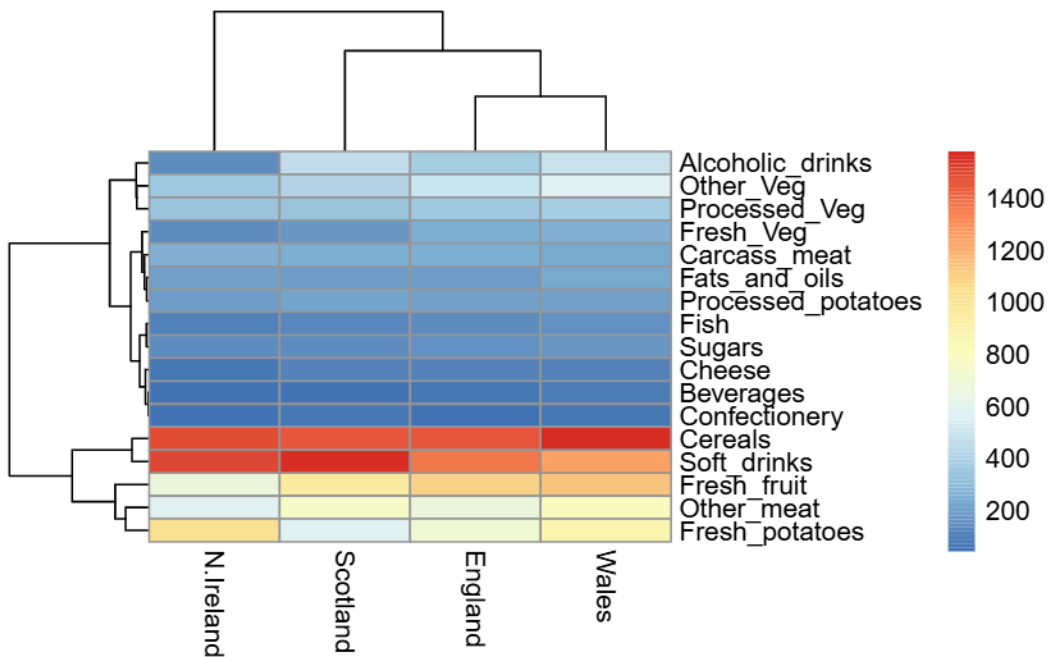
The `pairs()` function creates scatterplots comparing each pair of countries, where each point represents a food category. If a point lies on the diagonal, it means the consumption values for that food are equal (or very similar) between the two countries.

```
pairs(x, col=rainbow(nrow(x)), pch=16)
```



```
library(pheatmap)

pheatmap(as.matrix(x))
```



Q6. Based on the pairs and heatmap figures, which countries cluster together and what does this suggest about their food consumption patterns? Can you easily tell what the main differences between N. Ireland and the other countries of the UK in terms of this data-set?

England, Wales, and Scotland cluster together, showing similar consumption patterns. N. Ireland is more distinct. While we can see it differs overall, the exact food differences are not immediately clear from these plots alone.

Tidy the data

Fix anything that went wrong with import

Exploratory analysis

Make some plots to help make sense of obvious trends...

PCA to the rescue

The main function in “base” R for PCA is called `prcomp()`. This function wants the “observations” to be rows and the “variables” to be columns.

So here we need to take the transpose of our `x` input object.

```
# Use the prcomp() PCA function
pca <- prcomp( t(x) )
summary(pca)
```

Importance of components:

	PC1	PC2	PC3	PC4
Standard deviation	324.1502	212.7478	73.87622	3.176e-14
Proportion of Variance	0.6744	0.2905	0.03503	0.000e+00
Cumulative Proportion	0.6744	0.9650	1.00000	1.000e+00

so here we need to take the transpose of our `x` input object

```
pca <- prcomp( t(x) )
summary(pca)
```


Importance of components:

	PC1	PC2	PC3	PC4
Standard deviation	324.1502	212.7478	73.87622	3.176e-14
Proportion of Variance	0.6744	0.2905	0.03503	0.000e+00
Cumulative Proportion	0.6744	0.9650	1.00000	1.000e+00

The returned `pca` object has components we can use to make our main result figures:

```
attributes(pca)
```

```
$names
```

```
[1] "sdev"      "rotation" "center"   "scale"    "x"
```

```
$class
```

```
[1] "prcomp"
```

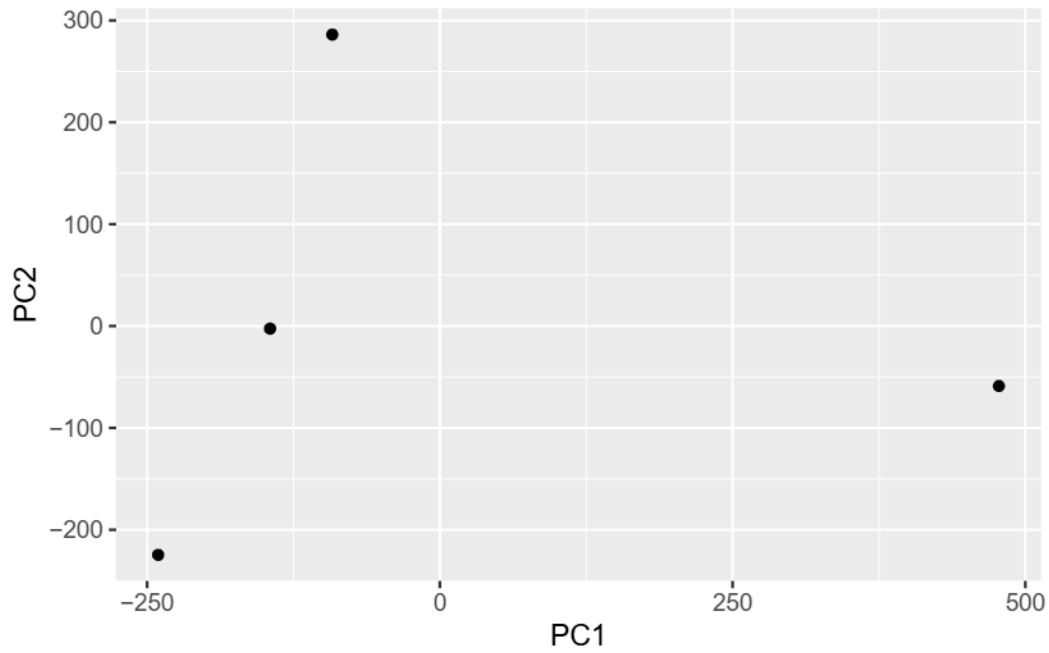
The main result figure from this analysis is called a “**score plot**” (a.k.a an “ordination plot”, “PC plot” or simply “PC1 vs PC2 plot”).

This plot shows how samples (in this case countries) relate to each other along our new PC axis.

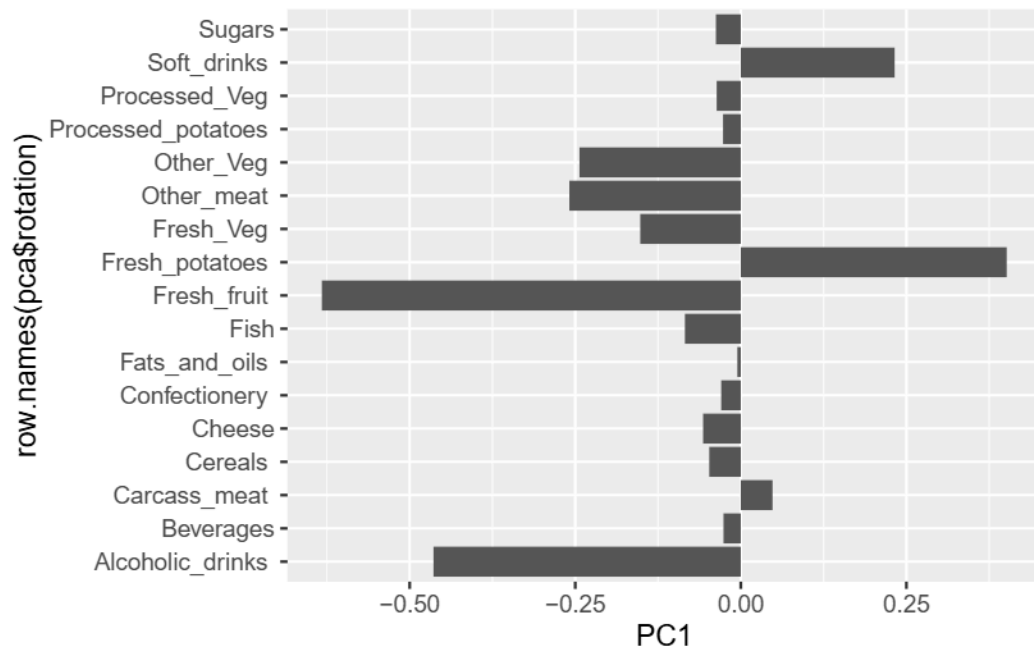
This is our new “reduced-dimensional space”. In this case 2 dimensions, PC1 and PC2, that capture most of the variance in the original 17 dimensional data-set.

```
library(ggplot2)
```

```
ggplot(pca$x) +  
  aes(PC1, PC2) +  
  geom_point()
```

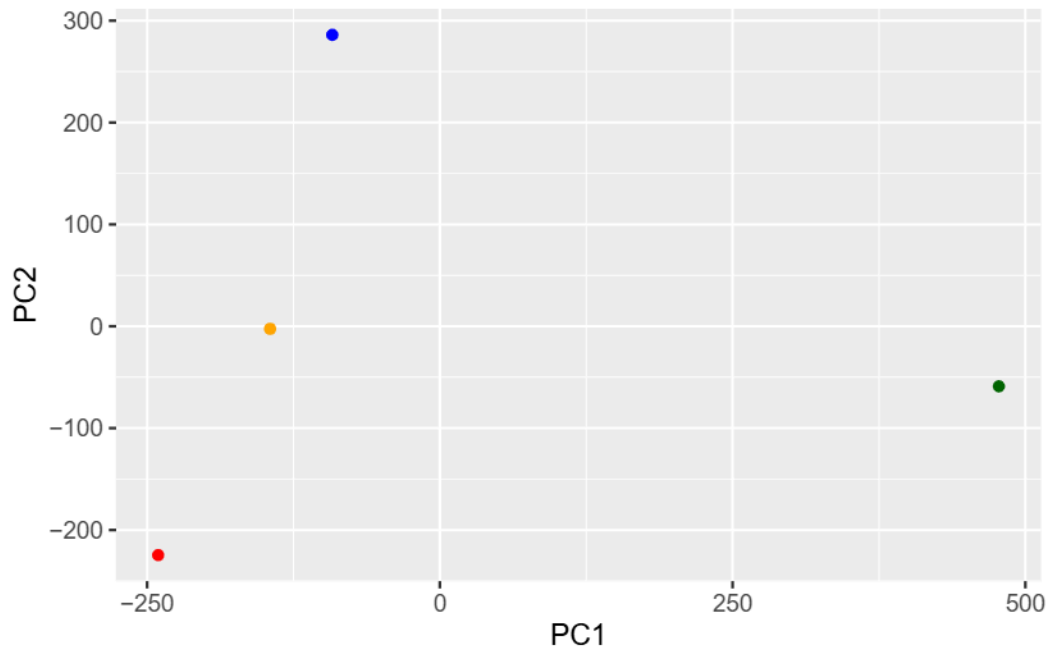


```
ggplot(pca$rotation) +  
  aes(PC1, row.names(pca$rotation)) +  
  geom_col()
```

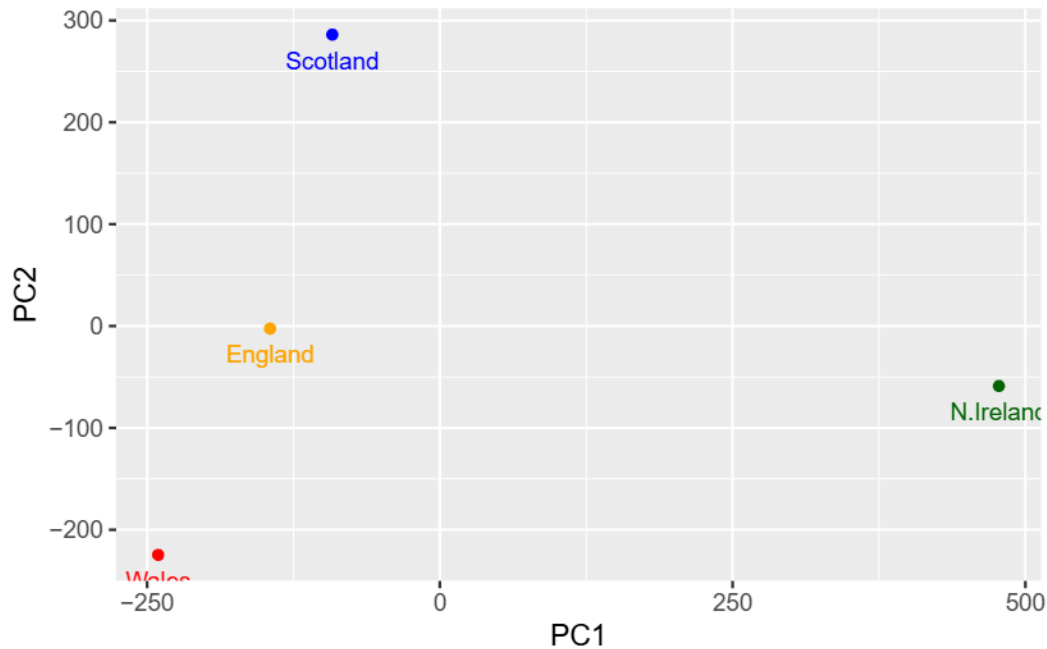


```
mycols <- c("orange", "red", "blue", "darkgreen")
```

```
ggplot(pca$x) +  
  aes(PC1, PC2) +  
  geom_point(col=mycols)
```



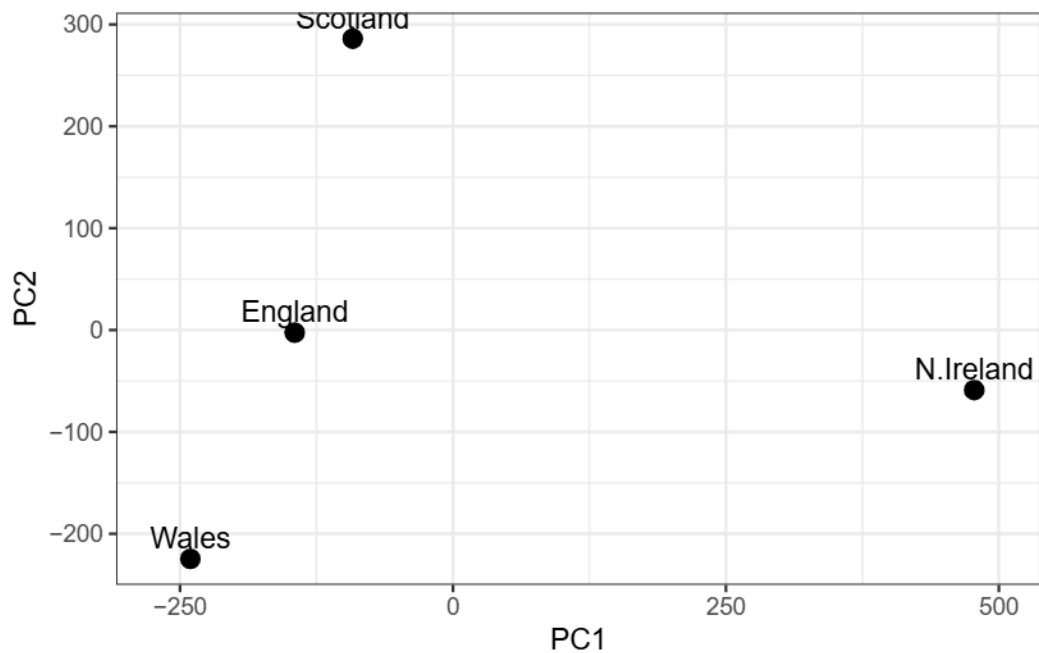
```
ggplot(pca$x) +  
  aes(PC1, PC2, label=row.names(pca$x)) +  
  geom_point(col=mycols) +  
  geom_text(size=3, vjust=2, col=mycols)
```



Q7. Complete the code below to generate a plot of PC1 vs PC2. The second line adds text labels over the data points.

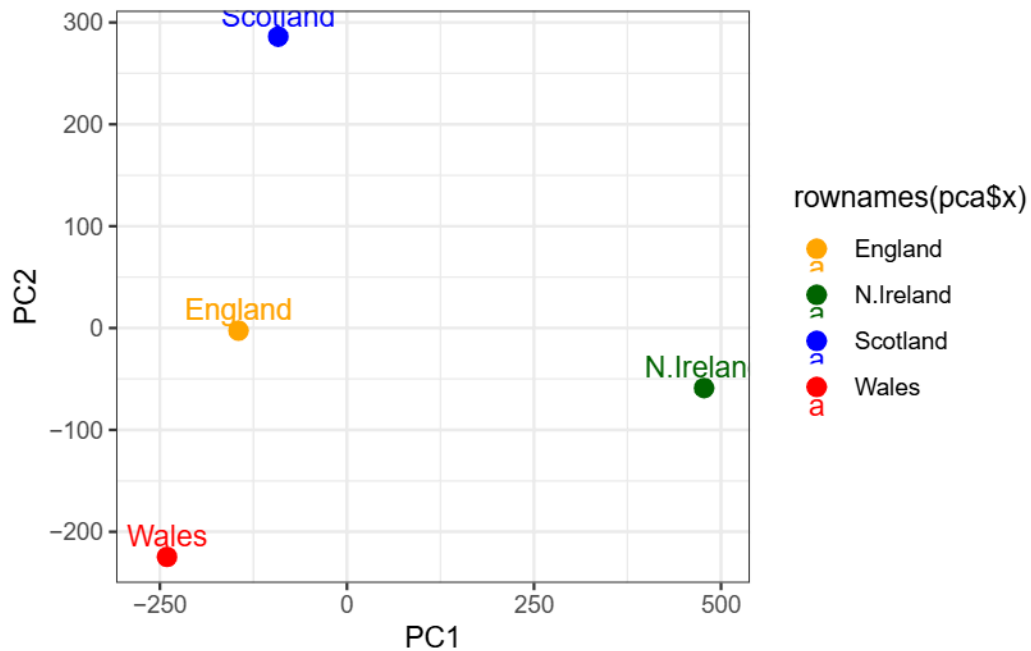
```
# Create a data frame for plotting
df <- as.data.frame(pca$x)
df$Country <- rownames(df)

# Plot PC1 vs PC2 with ggplot
ggplot(pca$x) +
  aes(x = PC1, y = PC2, label = rownames(pca$x)) +
  geom_point(size = 3) +
  geom_text(vjust = -0.5) +
  xlim(-270, 500) +
  xlab("PC1") +
  ylab("PC2") +
  theme_bw()
```



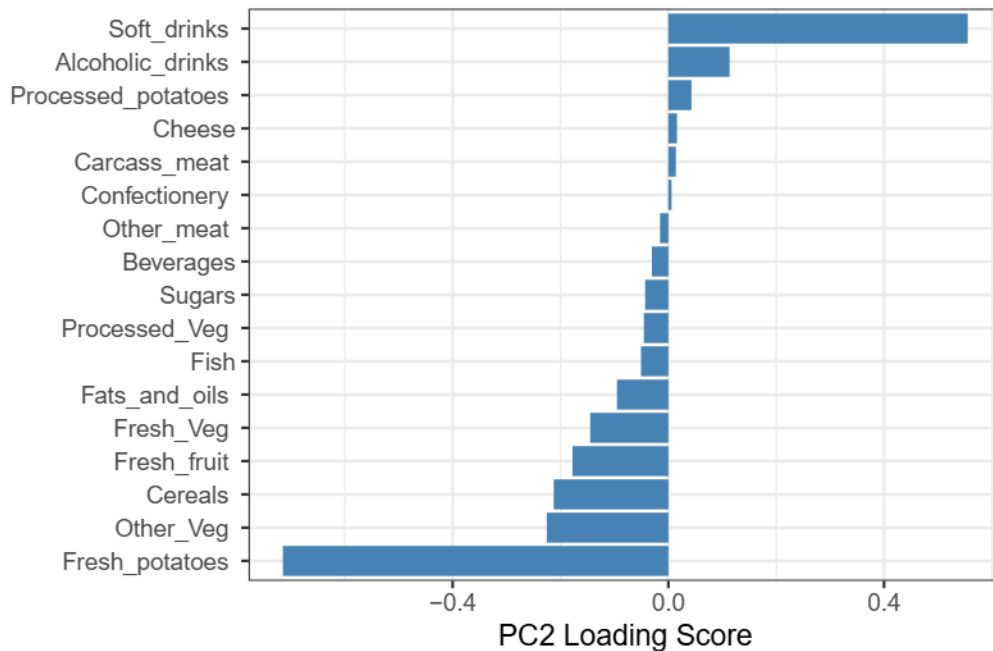
Q8. Customize your plot so that the colors of the country names match the colors in our UK and Ireland map and table at start of this document.

```
ggplot(pca$x) +
  aes(x = PC1, y = PC2, label = rownames(pca$x), color = rownames(pca$x)) +
  geom_point(size = 3) +
  geom_text(vjust = -0.5) +
  scale_color_manual(values = c(
    "England" = "orange",
    "Wales" = "red",
    "Scotland" = "blue",
    "N.Ireland" = "darkgreen"
  )) +
  xlim(-270, 500) +
  xlab("PC1") +
  ylab("PC2") +
  theme_bw()
```



Q9. Generate a similar ‘loadings plot’ for PC2. What two food groups feature prominently and what does PC2 mainly tell us about?

```
ggplot(pca$rotation) +
  aes(x = PC2, y = reorder(rownames(pca$rotation), PC2)) +
  geom_col(fill = "steelblue") +
  xlab("PC2 Loading Score") +
  ylab("") +
  theme_bw() +
  theme(axis.text.y = element_text(size = 9))
```



Fresh_veg and Fresh_fruit (strong loadings on one side) vs Soft_drinks (and sometimes Processed foods) on the opposite side. PC2 mainly represents a contrast between healthier fresh foods (fruits & vegetables) and less healthy / processed items (like soft drinks).

The score plot shows how samples are positioned along principal components, while the loading plot shows which variables influence those components. The loadings explain why samples appear where they do in the score plot.

Q10. How many genes and samples are in this data set? How many PCs do you think it will take to have a useful overview of this data set (see below)?

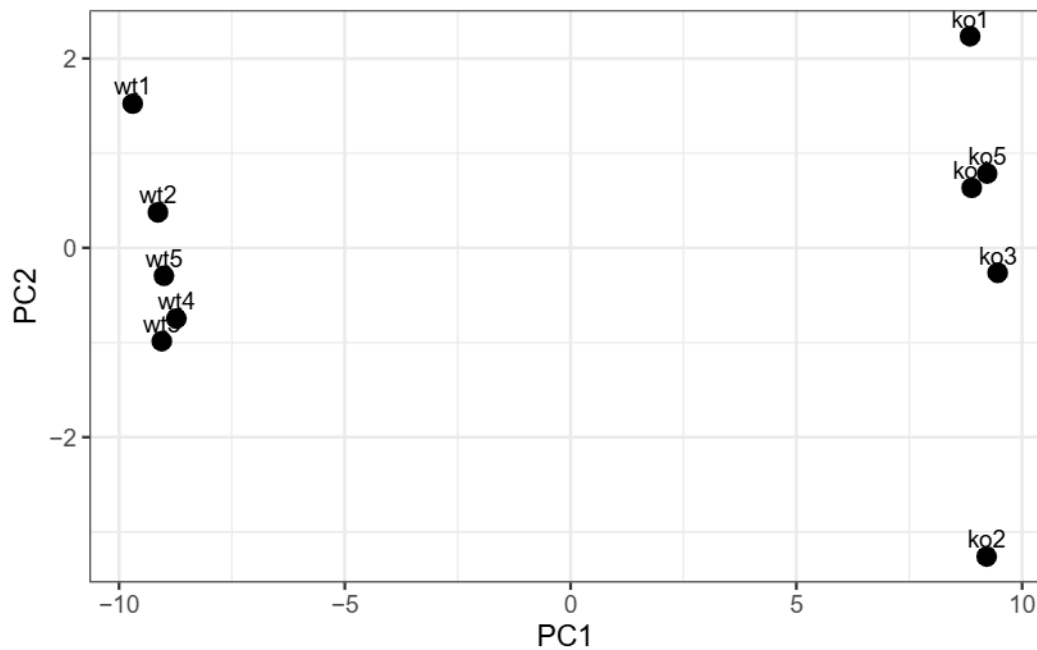
```
url2 <- "https://tinyurl.com/expression-CSV"
rna.data <- read.csv(url2, row.names=1)
head(rna.data)
```

	wt1	wt2	wt3	wt4	wt5	ko1	ko2	ko3	ko4	ko5
gene1	439	458	408	429	420	90	88	86	90	93
gene2	219	200	204	210	187	427	423	434	433	426
gene3	1006	989	1030	1017	973	252	237	238	226	210
gene4	783	792	829	856	760	849	856	835	885	894
gene5	181	249	204	244	225	277	305	272	270	279
gene6	460	502	491	491	493	612	594	577	618	638

```
## Again we have to take the transpose of our data
pca <- prcomp(t(rna.data), scale=TRUE)

# Create data frame for plotting
df <- as.data.frame(pca$x)
df$Sample <- rownames(df)

## Plot with ggplot
ggplot(df) +
  aes(x = PC1, y = PC2, label = Sample) +
  geom_point(size = 3) +
  geom_text(vjust = -0.5, size = 3) +
  xlab("PC1") +
  ylab("PC2") +
  theme_bw()
```



The dataset has 100 genes and 10 samples. About 1–2 principal components are sufficient, since PC1 explains ~92.6% of the variance and PC2 adds a small amount more.